CTS Interview Preparation Guide

Java | Spring Boot | Agile | Design Patterns | Collections | Threads | Maven | Git

1. Agile & Scrum

Q1. What is Agile?

Agile is an iterative software development methodology that delivers working software in small, frequent releases called **iterations/sprints**.

It focuses on collaboration, customer feedback, and flexibility over rigid planning.

Core values (Agile Manifesto): Individuals & interactions, Working software, Customer collaboration, Responding to change.

Q2. Difference between Agile and Waterfall

Agile: Iterative, flexible, continuous delivery, welcomes change, customer involved throughout.

Waterfall: Linear/sequential, rigid phases (Req → Design → Dev → Test → Deploy), change is costly, customer involved only at start and end.

Key difference: Agile delivers working software every sprint; Waterfall delivers at the end.

Q3. Benefits of Agile

• Faster delivery of working software • Early detection of bugs • Continuous customer feedback

• Better adaptability to changing requirements • Higher team collaboration • Reduced risk

Q4. What is a Story?

A **User Story** is a short, simple description of a feature from the end user's perspective.

Format: *"As a [user], I want to [action] so that [benefit]."*

Example: *"As a customer, I want to reset my password so that I can regain access."*

Q5. What is a Story Point?

A **Story Point** is a unit of measure for estimating the effort/complexity of a user story.

It is relative, not time-based. Teams use Fibonacci numbers (1, 2, 3, 5, 8, 13...) to estimate.

Higher points = more complex or uncertain work.

Q6. What is Scrum?

Scrum is an Agile framework for managing and completing complex work.

Key roles: Product Owner, Scrum Master, Development Team.

Key events: Sprint Planning, Daily Standup, Sprint Review, Sprint Retrospective.

Artifacts: Product Backlog, Sprint Backlog, Increment.

Q7. What is an Iteration?

An **iteration** is a fixed time-box (usually 1–4 weeks) in which the team completes a set of planned user stories and delivers a potentially shippable product increment.

Q8. What is a Sprint?

A **Sprint** is the Scrum term for an iteration — a fixed time-box (1–4 weeks) where the team works on selected backlog items and delivers a working increment.

Q10. What is a Burndown Chart?

A **Burndown Chart** is a graphical representation showing remaining work (story points) vs. time in a sprint.

X-axis = days in sprint, Y-axis = remaining story points.

Ideal line goes from top-left to bottom-right. If actual line is above ideal → team is behind.

Q11. What is a Standup Meeting?

A **Daily Standup** is a 15-minute daily meeting where each team member answers 3 questions:

1. What did I do yesterday? 2. What will I do today? 3. Are there any blockers?

Purpose: quick sync, identify impediments, not a status report to the manager.

2. Design Principles & Patterns

Q12. What is SOLID Principles?

S – Single Responsibility: A class should have only one reason to change.

O – Open/Closed: Open for extension, closed for modification.

L – Liskov Substitution: Subclasses must be substitutable for their base class.

I – Interface Segregation: Don't force a class to implement interfaces it doesn't use.

D – Dependency Inversion: Depend on abstractions, not concrete implementations.

Q13. What are Design Principles?

Design principles are guidelines for writing clean, maintainable, and scalable code.

Key principles: SOLID, DRY (Don't Repeat Yourself), KISS (Keep It Simple), YAGNI (You Aren't Gonna Need It), Separation of Concerns.

Q15. What is a Design Pattern? Different types?

A **Design Pattern** is a reusable solution to a commonly occurring problem in software design.

3 Categories:

- **Creational** – Object creation: Singleton, Factory, Abstract Factory, Builder, Prototype.
- **Structural** – Object composition: Adapter, Decorator, Facade, Proxy, Composite.
- **Behavioral** – Object interaction: Observer, Strategy, Command, Iterator, Template Method.

Q176. What is Facade Design Pattern? (used in SLF4J/Logger)

Facade provides a **simplified interface** to a complex subsystem.

SLF4J is a logging facade — it provides a single API, and you plug in any logging implementation (Log4j, Logback, etc.) behind it.

Benefit: You can swap the logging library without changing application code.

3. Java Core – Features, JVM, Memory

Q16. Features of Java?

- **Platform Independent** (Write Once Run Anywhere – via JVM)
- **Object-Oriented • Strongly Typed • Automatic Memory Management** (GC)
- **Multithreaded • Secure • Robust** (exception handling, no pointers)
- **Distributed • High Performance** (JIT compiler)

Q17. How is memory managed in Java?

Java memory is divided into:

- **Heap** – Objects are stored here; managed by **Garbage Collector (GC)**.
- **Stack** – Method calls, local variables (per thread).

• **Method Area** – Class metadata, static variables.

• **PC Register** – Current instruction per thread.

• **Native Method Stack** – Native (C/C++) method calls.

GC automatically reclaims heap memory of unreferenced objects.

Q20. What is ClassLoader? How does it work?

ClassLoader is a part of JVM that **loads .class files into memory** at runtime.

3 built-in ClassLoaders:

1. **Bootstrap** – loads core Java classes (java.lang, etc.) from rt.jar.

2. **Extension** – loads classes from ext directory.

3. **Application** – loads classes from classpath.

Process: Load → Link (Verify + Prepare + Resolve) → Initialize.

Q21. What is Bytecode and how does JVM interpret it?

After compilation, Java source code (.java) is converted to **Bytecode (.class)** — an intermediate, platform-neutral code.

JVM interprets bytecode and converts it to machine-specific code.

JIT (Just-In-Time) Compiler further optimizes by compiling frequently-used bytecode to native machine code at runtime.

Q22. When to use JDK vs JRE?

JDK (Java Development Kit) – Use when **developing** Java apps. Contains JRE + compiler (javac) + tools.

JRE (Java Runtime Environment) – Use when **running** Java apps. Contains JVM + libraries only.

Q26. What is a JAR file?

A **JAR (Java ARchive)** is a compressed file (.jar) that bundles .class files, resources, and metadata (MANIFEST.MF) into one package.

Benefits: Easy distribution, versioning, can be executed directly, used as libraries.

Create: `jar cf myapp.jar *.class`

Unjar: `jar xf myapp.jar`

4. Data Types & Strings

Q30. Data types in Java

Primitive: byte, short, int, long, float, double, char, boolean

Non-Primitive (Reference): String, Array, Class, Interface, Enum

Q31. Primitive Data Types

byte(1B), short(2B), int(4B), long(8B), float(4B), double(8B), char(2B), boolean(1bit)

Q33. What is String?

String is an **immutable** sequence of characters in Java.

Once created, its value cannot be changed. Stored in the **String Pool**.

Example: `String s = "Hello";`

Q34. What is StringBuffer?

StringBuffer is a mutable sequence of characters. **Thread-safe** (synchronized methods).

Use when multiple threads modify the same string.

Q35. What is StringBuilder? Difference from StringBuffer?

StringBuilder is also mutable but **NOT thread-safe**.

Faster than StringBuffer in single-threaded scenarios.

| | String | StringBuffer | StringBuilder |

| Mutable | No | Yes | Yes |

| Thread-safe | Yes (immutable) | Yes | No |

| Performance | Slow for concat | Medium | Fastest |

Q36. What is String Pool?

String Pool is a special memory area in the Heap where String literals are stored.

When you create `String s = "Hello"`, JVM checks pool first. If found, reuses same reference. Saves memory.

`new String("Hello")` always creates a new object outside the pool.

Q37. Scenario for String vs StringBuffer

Use **String** when the value doesn't change (e.g., storing a name, key).

Use **StringBuffer** in multi-threaded apps that modify strings (e.g., logs).

Use **StringBuilder** in single-threaded apps building strings in a loop.

5. OOP Concepts – Classes, Inheritance, Polymorphism

Q42. What is a Constructor?

A constructor is a special method called when an object is created. Same name as class, no return type.

Types: Default constructor, Parameterized constructor, Copy constructor.

Q43. What is Copy Constructor?

A constructor that creates a new object by copying values from an existing object.

Java doesn't provide it by default; you write it manually.

Example:

```
class Car {
  String name;
  Car(Car c) { this.name = c.name; }  // copy constructor
}
```

Q45. What is clone() in Object class?

clone() creates an exact copy of an object.

The class must implement the **Cloneable** interface; otherwise CloneNotSupportedException is thrown.

Shallow copy by default. For deep copy, override clone() and clone nested objects too.

Q49. Methods of Object class

toString(), equals(), hashCode(), clone(), getClass(), wait(), notify(), notifyAll(), finalize()

Q50. Difference between hashCode() and equals()

equals() – checks logical equality of two objects.

hashCode() – returns an integer hash used in hashing-based collections (HashMap, HashSet).

Contract: If equals() returns true, hashCode() MUST be the same. Override both together.

Q52. What is static?

Static belongs to the **class**, not an instance. Shared across all objects.

- **Static variable** – shared by all instances.
- **Static method** – called on class, can't use 'this' or instance vars.
- **Static block** – executed once when class is loaded.

• **Static class** – only nested classes can be static.

Q53. What is a static block? When is it executed?

A static block is executed **once** when the class is first loaded into memory by the ClassLoader.

Used for static initialization (e.g., loading DB drivers).

Runs **before** the main() method and constructors.

```
class Demo {
  static { System.out.println("Class loaded!"); }
  public static void main(String[] a) { System.out.println("main"); }
}
```

Q54. What is Abstract Class?

An abstract class is a class that **cannot be instantiated** and may have abstract (unimplemented) and concrete methods.

Declared with `abstract` keyword.

Q55. Abstract class vs Normal class

Abstract: Cannot instantiate, can have abstract methods, forces subclass to implement.

Normal: Can instantiate, all methods have body.

Q56. Abstract class vs Interface

Abstract Class: Can have state (instance variables), constructors, concrete methods, single inheritance.

Interface: No state (only constants), no constructors, all methods abstract by default (Java 8+ allows default/static), supports multiple inheritance.

Q58. Access Modifiers

• **private** – same class only.

• **default** (no keyword) – same package.

• **protected** – same package + subclasses.

• **public** – everywhere.

Q62. What is Inheritance?

Inheritance allows a class (child) to **acquire properties and methods** of another class (parent) using `extends`.

Types: Single, Multilevel, Hierarchical, Multiple (via interfaces), Hybrid.

Q64. What is the Diamond Problem?

When class D extends two classes B and C (both extending A), and B and C override A's method, there's ambiguity — which version does D inherit?

Java solves this by **NOT allowing multiple inheritance of classes**. Only allowed via interfaces (where Java 8 requires explicit override if conflict exists).

Q66. What is Method Overloading?

Defining multiple methods with the **same name but different parameters** (type, number, or order) in the same class.

Resolved at **compile time** (static polymorphism).

Example: `add(int a, int b)` and `add(double a, double b)`

Q70. What is Method Overriding?

A subclass provides a **specific implementation** of a method already defined in its parent class.

Same method name, same parameters. Resolved at **runtime** (dynamic polymorphism).

Requires `@Override` annotation (best practice).

Q67. IS-A vs HAS-A Relationship

IS-A – Inheritance. Dog IS-A Animal. Implemented via `extends`.

HAS-A – Composition/Aggregation. Car HAS-A Engine. Implemented by having an object as a field.

Q75. Why getters and setters?

To achieve **encapsulation** — hide internal state and control access.

Getters read private fields. Setters write with validation.

Example: setter can validate age > 0 before setting.

6. Exception Handling

Q77. What is Exception Handling?

A mechanism to handle **runtime errors** gracefully without crashing the program.

Keywords: try, catch, finally, throw, throws.

Q78. Types of Exceptions

Checked Exceptions – Checked at compile time. Must be handled or declared.

Unchecked Exceptions – Checked at runtime. Subclasses of RuntimeException.

Q79. Checked Exceptions

IOException, FileNotFoundException, SQLException, ClassNotFoundException, ParseException, InterruptedException

Q80. Unchecked Exceptions

NullPointerException, ArrayIndexOutOfBoundsException, ArithmeticException, ClassCastException, NumberFormatException, StackOverflowError

Q81. Exception vs Error

Exception – Recoverable. Application can handle it (e.g., IOException).

Error – Unrecoverable. JVM-level problems (e.g., OutOfMemoryError, StackOverflowError). Don't try to catch Errors.

Q82. throw vs throws

throw – Used inside a method to explicitly throw an exception instance.

throws – Used in method signature to declare that the method might throw a checked exception.

```
throw new IOException("File not found");
```

```
public void read() throws IOException { }
```

Q84. Exception Hierarchy

Throwable

■■■ Error (OutOfMemoryError, StackOverflowError)

■■■ Exception

■■■ Checked (IOException, SQLException, ...)

■■■ RuntimeException (NullPointerException, ArithmeticException, ...)

Q86. Can one try have multiple catches?

Yes! You can have multiple catch blocks. Order matters — more specific exceptions first, broader ones last.

Q87 Scenario: Having catch(Exception), catch(RuntimeException), catch(IOException) in wrong order causes a compile error because RuntimeException and IOException are subclasses of Exception. **Eclipse will flag 'Unreachable catch block'** — specific catches must come before general ones.

Q83. How to create a Custom Exception?

```
class InsufficientFundsException extends Exception {
  public InsufficientFundsException(String msg) {
    super(msg);
  }
}

// Usage
throw new InsufficientFundsException("Balance too low");
```

7. Collections Framework

Q94. Topmost class/interface of Collections?

The root interface is **java.util.Collection**.

Above that is **java.lang.Iterable**.

Map is separate — it does NOT extend Collection.

Q95. Store unique records in HashSet

Override **hashCode()** and **equals()** in your object class.

HashSet uses hashCode to find bucket, then equals to check duplicates.

If both return consistent results for logically equal objects, duplicates are rejected.

Q98. Use of Buckets in HashMap

A bucket is a slot in the internal array of HashMap.

When you put a key, its hashCode determines the bucket index. Multiple keys in the same bucket form a linked list (or tree in Java 8+).

Goal: distribute entries evenly to minimize collisions and achieve O(1) lookup.

Q100. Comparable vs Comparator

Comparable – Natural ordering. Class implements `Comparable<T>` and overrides `compareTo()`. Modifies the class.

Comparator – Custom ordering. Separate class/lambda implements `Comparator<T>` and overrides `compare()`. Doesn't modify class.

compareTo() returns: 0 if equal, positive if this > other, negative if this < other.

Q102. What is HashMap?

HashMap is a key-value pair data structure. Key must be unique; values can repeat.

Backed by an array + linked list. Key's hashCode determines storage location.

Allows one null key and multiple null values. NOT thread-safe. Not ordered.

Q108. Synchronized HashMap vs ConcurrentHashMap

Synchronized HashMap: Wraps entire map with a lock. Only one thread at a time — low performance.

ConcurrentHashMap: Divides map into segments; each segment has its own lock — multiple threads can read/write concurrently. High performance in multi-threaded apps.

Q112. ArrayList vs LinkedList

ArrayList: Dynamic array. Fast random access O(1). Slow insert/delete in middle O(n).

LinkedList: Doubly linked list. Slow random access O(n). Fast insert/delete O(1).

Use ArrayList for frequent reads. **Use LinkedList** for frequent add/remove operations.

Q113. Use of TreeSet

TreeSet stores elements in **sorted order** (natural or custom comparator). No duplicates.

To sort objects: implement Comparable in your class, or provide a Comparator to TreeSet constructor.

Q115. TreeSet vs TreeMap

TreeSet – Stores unique values in sorted order.

TreeMap – Stores key-value pairs sorted by key.

Both use a Red-Black tree internally. O(log n) for add/remove/search.

Q117. ArrayList vs Vector

ArrayList: Not synchronized, faster, modern preferred choice.

Vector: Synchronized (thread-safe), slower, legacy class.

Q130. Duplicate key in HashMap?

The **old value is replaced** by the new value for the same key. Key remains unique.

8. Threads & Concurrency

Q118. How to create a thread in Java?

1. Extend Thread class:

```
class MyThread extends Thread {
  public void run() { System.out.println("Thread running"); }
}
new MyThread().start();
```

2. Implement Runnable:

```
Runnable r = () -> System.out.println("Runnable running");
new Thread(r).start();
```

Q120. Life Cycle of a Thread

NEW – Thread created, not yet started.

RUNNABLE – start() called; ready to run / running.

BLOCKED – Waiting to acquire a lock.

WAITING – Waiting indefinitely (wait(), join()).

TIMED_WAITING – Waiting for a specified time (sleep(ms), wait(ms)).

TERMINATED – run() method completed.

Q121. wait() vs sleep()

wait() – Releases the lock; another thread must call notify() to wake it. Called on object.

sleep() – Does NOT release the lock; thread pauses for specified time then resumes. Called on Thread.

Q125. What is Synchronization?

Synchronization ensures that only one thread accesses a shared resource at a time, preventing data inconsistency.

Done using the **synchronized** keyword on methods or blocks.

Q126. What is ExecutorService?

ExecutorService is a higher-level API for managing thread pools.

`ExecutorService es = Executors.newFixedThreadPool(5);`

`es.submit(() -> doTask());`

Advantages: reuse threads, manage lifecycle, handle Future results.

Q127. What is Thread Pool?

A thread pool is a collection of pre-created, reusable threads.

Eliminates overhead of creating/destroying threads for every task. Controlled via ExecutorService.

Q128. Runnable vs Callable

Runnable: run() returns void. Cannot throw checked exceptions.

Callable: call() returns a value (Future<T>). Can throw checked exceptions.

9. Java 8 Features

Q131. Java 8 Features

Lambda Expressions, Functional Interfaces, Stream API, Optional Class, Default/Static methods in interfaces, Method References, New Date/Time API (LocalDate, LocalTime), Nashorn JS Engine, forEach() in Iterable.

Q132. What is a Functional Interface?

An interface with **exactly one abstract method**. Annotated with `@FunctionalInterface`.

Examples: Runnable, Callable, Comparator, Consumer, Predicate, Function, Supplier.

Q133. Consumer, Producer, Function, Predicate

```
// Consumer - takes input, no return
Consumer print = s -> System.out.println(s);
print.accept("Hello");

// Predicate - takes input, returns boolean
Predicate isEven = n -> n % 2 == 0;
System.out.println(isEven.test(4));  // true

// Function - takes input, returns output
Function len = s -> s.length();
System.out.println(len.apply("Java"));  // 4

// Supplier - no input, returns value
Supplier greet = () -> "Hello World";
System.out.println(greet.get());
```

Q134. Method Referencing

A shorthand for lambdas that call an existing method.

Types: Static (`ClassName::method`), Instance (`obj::method`), Constructor (`Class::new`).

Example: `list.forEach(System.out::println);` instead of `list.forEach(x -> System.out.println(x));`

Q138. What are Streams?

Streams provide a **functional, pipeline-style** way to process collections of data.

Operations:

• **Intermediate:** filter(), map(), sorted(), distinct(), limit(), skip()

• **Terminal:** collect(), forEach(), count(), reduce(), findFirst()

```
List nums = Arrays.asList(1,2,3,4,5,6);
List evens = nums.stream()
    .filter(n -> n % 2 == 0)
    .collect(Collectors.toList());  // [2, 4, 6]
```

Q140. What is Optional Class?

Optional is a container that may or may not contain a non-null value. Avoids NullPointerException.

Methods: of(), ofNullable(), empty(), isPresent(), get(), orElse(), orElseGet(), ifPresent(), map(), filter()

```
Optional name = Optional.ofNullable(getName());
String result = name.orElse("Unknown");
```

Q142. LocalDate and LocalTime in Java 8

LocalDate – Represents date only (yyyy-MM-dd). No time, no timezone.

LocalTime – Represents time only (HH:mm:ss).

LocalDateTime – Both date and time.

Immutable and thread-safe. From java.time package.

Q143. Date vs LocalDate

java.util.Date – Mutable, not thread-safe, poor API, handles time and timezone confusingly.

java.time.LocalDate – Immutable, thread-safe, clean API, no timezone baggage. Preferred in modern Java.

Q179. What is Parallel Streams?

Parallel streams split data into multiple chunks and process them **concurrently** using multiple CPU cores (ForkJoinPool).

`list.parallelStream().filter(...).collect(...)`

Use when: Large datasets, CPU-intensive operations, order doesn't matter.

Avoid when: Small datasets, order matters, tasks have side effects.

10. Modern Java – Records, Sealed, Generics, Reflection

Q144. What is Sealed Class?

A sealed class restricts which classes can extend it.

Declared with `sealed` keyword; permitted subclasses listed with `permits`.

Useful for modeling restricted hierarchies (e.g., payment types).

```
sealed class Shape permits Circle, Rectangle { }
final class Circle extends Shape { }
final class Rectangle extends Shape { }
```

Q145. What is Record?

A **Record** (Java 14+) is a concise, immutable data carrier class. Auto-generates constructor, getters, equals(), hashCode(), toString().

```
record Person(String name, int age) { }

Person p = new Person("Alice", 30);
System.out.println(p.name());  // Alice
```

Q147. What is DTO?

DTO (Data Transfer Object) is a simple object used to transfer data between layers (e.g., Controller → Service → Client).

No business logic. Only fields + getters/setters (or a Record in modern Java).

Q148. What is DAO?

DAO (Data Access Object) is a pattern that provides an abstraction layer for database operations (CRUD).

Separates persistence logic from business logic.

Q89. What is Generics in Java?

Generics allow classes, interfaces, and methods to operate on **typed parameters**, providing type safety at compile time.

```
class Box {
  T value;
  Box(T v) { this.value = v; }
  T get() { return value; }
}

Box intBox = new Box<>(42);
Box  strBox = new Box<>("Hello");
```

Q109. What is Java Reflection?

Reflection allows inspection and modification of class behavior at **runtime**.

Can get class info, invoke methods, access private fields dynamically.

Used in frameworks (Spring, Hibernate) for dependency injection, ORM mapping.

```
Class<?> c = obj.getClass();
Method m = c.getMethod("setName", String.class); m.invoke(obj, "Alice");
```

11. Maven

Q150. What is Maven?

Maven is a **build automation and dependency management tool** for Java projects.

Uses **POM.xml** to declare project configuration, dependencies, and build lifecycle.

Q151. Full form of POM.xml

POM = Project Object Model. An XML file that contains project info, dependencies, plugins, and build config.

Q152. Content of POM.xml

groupId, artifactId, version, packaging, dependencies, plugins, build config, properties, repositories

Q153. groupId vs artifactId

groupId – Organization/package namespace (e.g., com.cognizant).

artifactId – Project name (e.g., employee-service).

Together they uniquely identify the project in Maven repository.

Q155. Dependency Scope in Maven

compile – Default. Available at compile, test, and runtime.

test – Only for test compilation and running (e.g., JUnit).

provided – Available at compile time, NOT packaged (e.g., Servlet API provided by server).

runtime – Not needed for compile, but needed at runtime (e.g., JDBC driver).

system – Like provided but specify explicit path.

Q160. Maven Commands

mvn validate – Validate POM is correct.

mvn compile – Compile source code.

mvn test – Run unit tests.

mvn package – Package into JAR/WAR.

mvn install – Install to local .m2 repository.

mvn clean – Delete target directory.

mvn clean install – Clean + full build.

mvn deploy – Deploy to remote repository.

Q159. Which site does Maven download from?

By default, Maven downloads from **https://repo.maven.apache.org/maven2** (Maven Central Repository).

Organizations use private repos like Nexus or JFrog Artifactory.

12. Git Commands

Q178. 10 Key Git Commands

git init – Initialize a new local repository.

git clone <url> – Copy a remote repository locally.

git add . – Stage all changes for commit.

git commit -m "msg" – Save staged changes with a message.

git push – Upload local commits to remote (GitHub/GitLab).

git pull – Fetch + merge changes from remote.

git branch <name> – Create a new branch.

git checkout <branch> – Switch to a branch.

git merge <branch> – Merge a branch into current branch.

git status – Show working directory and staging status.

Bonus: git log – View commit history.

Bonus: git stash – Temporarily save uncommitted changes.

13. Reactive Programming – Mono, Flux, Reactor

Q180. What is Mono and Flux?

Both are from **Project Reactor** (used in Spring WebFlux).

Mono<T> – Emits **0 or 1** element asynchronously.

Flux<T> – Emits **0 to N** elements asynchronously.

Example: `Mono.just("Hello")`, `Flux.fromIterable(list)`

Q181. What is Asynchronous Programming?

Asynchronous programming allows tasks to execute **without blocking** the calling thread.

The caller continues executing; a callback/handler is triggered when the async task completes.

Example: Making a DB call without waiting — meanwhile handle other requests.

Q182. What is Reactor?

Project Reactor is a reactive library for building **non-blocking, event-driven** applications on the JVM.

Implements the Reactive Streams specification. Used by Spring WebFlux.

Q183. What is Backpressure?

Backpressure is a mechanism where the **consumer signals the producer** to slow down if it's sending data too fast.

Prevents overwhelming the subscriber with more data than it can handle.

Reactor supports backpressure strategies: BUFFER, DROP, LATEST, ERROR.

14. Spring Framework & Spring Boot

Key Spring Interview Questions

QS1. What are the key features of the Spring Framework?

IoC/DI, AOP, MVC, Data Access (JDBC/ORM), Transaction Management, Security, Testing support, Spring Boot for auto-config.

QS2. What is Spring Container?

The Spring IoC Container manages beans — their creation, wiring, lifecycle.

Types: **BeanFactory** (basic, lazy) and **ApplicationContext** (full-featured, eager, preferred).

QS3. What is Dependency Injection?

DI is a design pattern where Spring **injects dependencies** into a class instead of the class creating them.

Types: Constructor Injection (recommended), Setter Injection, Field Injection (@Autowired).

QS4. BeanFactory vs ApplicationContext

BeanFactory: Basic container. Lazy initialization. Minimal features.

ApplicationContext: Advanced container. Eager loading. Supports events, AOP, i18n, @Annotations. Preferred.

QS5. Scopes of a Spring Bean

singleton – Default. One instance per Spring container.

prototype – New instance every time the bean is requested.

request – One per HTTP request (web apps).

session – One per HTTP session.

application – One per ServletContext.

QS6. Constructor Injection vs Setter Injection

Constructor: Dependencies injected via constructor. Immutable, recommended for mandatory deps.

Setter: Dependencies injected via setter methods. Flexible for optional deps.

QS7. Spring Bean Lifecycle

1. Instantiate → 2. Populate Properties → 3. BeanNameAware/BeanFactoryAware → 4. @PostConstruct / afterPropertiesSet() → 5. Bean Ready → 6. @PreDestroy / destroy() → 7. Destroyed

QS8. Spring Boot vs Spring Framework

Spring Framework: Core framework. Manual configuration (XML or Java).

Spring Boot: Built on Spring. Auto-configuration, embedded server (Tomcat), starter dependencies, production-ready with Actuator. Minimal boilerplate.

QS9. Eager Loading vs Lazy Loading

Eager Loading: Beans/data loaded at startup. Default for singleton beans.

Lazy Loading: Loaded only when first needed. Use `@Lazy` annotation or `fetch = FetchType.LAZY` in Hibernate.

QS10. @Component, @Service, @Repository, @Controller

All are specializations of `@Component` — they register beans in the Spring context.

@Component – Generic bean.

@Service – Business logic layer.

@Repository – Data access layer (also translates DB exceptions).

@Controller – Spring MVC controller (returns view names).

@RestController – @Controller + @ResponseBody (returns JSON/XML).

QS11. @Bean vs @Component

@Component – Auto-detected via classpath scanning. Annotate the class directly.

@Bean – Explicitly declared in a `@Configuration` class. Used for third-party classes you can't annotate.

QS12. @Qualifier annotation

When multiple beans of the same type exist, `@Qualifier("beanName")` tells Spring which one to inject.

Used alongside `@Autowired`.

QS13. @Value and @PropertySource

@Value("${property.key}") – Injects a value from application.properties into a field.

@PropertySource("classpath:custom.properties") – Loads an external properties file into Spring Environment.

QS14. @SpringBootApplication

A convenience annotation that combines:

@Configuration + **@EnableAutoConfiguration** + **@ComponentScan**

Marks the main class and triggers auto-configuration and component scanning.

QS15. Spring Boot Auto-configuration

Spring Boot reads META-INF/spring.factories (or spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports in Boot 3) and conditionally configures beans based on what's on the classpath.

Example: If H2 is on classpath, auto-configures an in-memory DataSource.

QS16. Spring Boot Starters

Starters are **curated dependency bundles** that pull in everything needed for a feature.

Examples: spring-boot-starter-web (MVC + Tomcat), spring-boot-starter-data-jpa (Hibernate + JPA), spring-boot-starter-security, spring-boot-starter-test.

QS17. How to create a REST API in Spring Boot?

```
@RestController
@RequestMapping("/api")
public class UserController {

  @GetMapping("/users")
  public List getUsers() {
    return userService.findAll();
  }

  @PostMapping("/users")
  public User createUser(@RequestBody User user) {
    return userService.save(user);
  }
}
```

QS18. application.properties vs application.yml

.properties: Key-value pairs. `server.port=8080`

.yml: Hierarchical YAML format. More readable for complex configs.

Both work the same; choose based on team preference. Profiles: application-dev.yml, application-prod.yml.

QS19. Global Exception Handling in Spring Boot

Use **@ControllerAdvice** + **@ExceptionHandler**:

```
@ControllerAdvice
public class GlobalExceptionHandler {
  @ExceptionHandler(ResourceNotFoundException.class)
  public ResponseEntity handle(ResourceNotFoundException ex) {
    return ResponseEntity.status(404).body(ex.getMessage());
  }
}
```

QS20. Spring Boot Actuator

Actuator provides **production-ready monitoring endpoints**: /actuator/health, /actuator/info, /actuator/metrics, /actuator/env.

Add `spring-boot-starter-actuator` dependency. Expose/secure endpoints via properties.

QS21. Spring MVC and DispatcherServlet

Spring MVC is the web framework in Spring following the MVC pattern.

DispatcherServlet is the **Front Controller** — all HTTP requests go through it.

Flow: Request → DispatcherServlet → HandlerMapping → Controller → ViewResolver → Response

QS22. @RequestMapping, @PathVariable, @RequestParam

@RequestMapping – Maps HTTP requests to controller methods.

@PathVariable – Extracts value from URL path: `/users/{id}`

@RequestParam – Extracts query parameter: `/users?page=1`

QS23. HandlerInterceptor

Intercepts requests before and after controller execution.

Methods: `preHandle()` (before), `postHandle()` (after), `afterCompletion()` (after view).

Used for logging, authentication, performance tracking.

QS24. JUnit & Testing Annotations

@BeforeEach – Runs before every test method (JUnit 5). @Before in JUnit 4.

@AfterEach – Runs after every test method. @After in JUnit 4.

@BeforeAll – Runs once before all tests in class.

@AfterAll – Runs once after all tests in class.

@MockBean – Creates a Mockito mock and adds it to Spring context (Spring Boot test).

@Mock – Creates a Mockito mock without Spring context (unit test).

Q175. What is Logger? What is SLF4J?

A Logger records application events/messages at different levels: TRACE, DEBUG, INFO, WARN, ERROR.

SLF4J (Simple Logging Facade for Java) is a logging abstraction — your code uses SLF4J API; at runtime you plug in an implementation (Logback, Log4j2).

Benefits: Easily switch logging implementation without changing code, better performance than System.out.println.

```
private static final Logger log = LoggerFactory.getLogger(MyClass.class);
log.info("User created: {}", userId);
log.error("Error occurred", exception);
```

Best of luck with your CTS Interview! You've got this! ■